HOMEWORK 6 — FINAL CAPSTONE

Transaction Processing Pipeline

A four-agent capstone workflow: Specification → Code Generation → Unit Tests → Documentation

Created by Yurii Habrusiev

OBJECTIVE

Problem & Objective

- Goal: validate, risk-score, and compliance-screen every transaction in `sample-transactions.json` (8 sample records), producing one auditable outcome per transaction — `REJECTED` / `CLEARED` / `HELD_FOR_REVIEW`.
- Non-negotiable domain rules:
  - Money is always `decimal.Decimal` — never `float`.
  - Currency validated against an offline ISO 4217 allow-list.
  - Sensitive fields (`source_account`, `destination_account`, `description`) never hit a log or console in plaintext.

META LEVEL

Four-Agent Pipeline

1

spec-writer

Wrote `specification.md` + `agents.md` from `TASKS.md`.

2

pipeline-builder

Built `orchestrator.py`, `pipeline/*.py`, `frontend/`, `mcp/server.py`, `.mcp.json`; logged 2+ context7 queries.

3

test-writer

Built `tests/` — 69 tests, 99% coverage on `pipeline/` + `mcp/`.

4

docs-writer

`README.md`, `HOWTORUN.md`, this presentation.

SYSTEM DESIGN

Architecture Overview

```
orchestrator.py -> shared/input/ -> Validation -> shared/output/->processing/
   -> Fraud Detection -> shared/output/->processing/ -> Compliance Check
   -> shared/results/ (terminal records + summary.json + audit_log.jsonl)
   -> consumed independently by frontend/ (static dashboard) and
      mcp/server.py ("pipeline-status" MCP server)
```

- Stages never call each other's code directly — every hand-off is a file move through `shared/{input,processing,output,results}/` using a standard message envelope (`message_id`, `timestamp`, `source_stage`, `target_stage`, `message_type`, `data`).
- Per-record isolation: one malformed record cannot crash the batch — exceptions are caught and written as `REJECTED` / `INTERNAL_ERROR`.

STAGE 1

Validation — `pipeline/validator.py`

- Confirms all 9 required fields are present (`MISSING_REQUIRED_FIELD`).
- Parses `amount` as `Decimal(str(...))` (`INVALID_AMOUNT_FORMAT` on failure).
- Enforces sign convention by `transaction_type` : `transfer` / `wire_transfer` must be positive (`NON_POSITIVE_AMOUNT`); `refund` must be negative (`REFUND_MUST_BE_NEGATIVE`).
- Validates `currency` against a fixed, offline 40-code ISO 4217 set (`CURRENCY_NOT_ISO4217`) — no network calls.
- Rejects terminate immediately, written straight to `shared/results/` ; everything else is forwarded, normalized, to Fraud Detection.

STAGE 2

Fraud Detection — `pipeline/fraud_detector.py`

- Never rejects on its own — only scores and annotates.
- Additive risk score (capped at 100): amount tier on `abs(amount)` (+60 / +40 / +15, mutually exclusive), off-hours UTC timestamp (<6 or ≥22, +20), cross-border destination (`metadata.country` outside `{"US"}` , +15, fails safe to cross-border if missing/unknown), wire-transfer channel (+10).
- Risk level: `LOW` (<30) / `MEDIUM` (30–59) / `HIGH` (≥60); `fraud_status` is `flagged` for anything not `LOW` .
- Always forwards to Compliance Check — the final decision is not made here.

STAGE 3

Compliance Check — `pipeline/compliance_checker.py`

- Terminal stage for every record that survived Validation.
- Watchlist screen on `source_account` / `destination_account` against a static set (`WATCHLIST_ACCOUNTS = {"ACC-9999"}`) — a hit overrides the fraud score entirely → `HELD_FOR_REVIEW` / `WATCHLIST_MATCH` , account masked (`ACC-***99`) in the human-facing note.
- Otherwise, anything the fraud stage flagged → `HELD_FOR_REVIEW` / `FRAUD_RISK_FLAGGED` .
- Everything else → `CLEARED` .
- Writes exactly one terminal record to `shared/results/<transaction_id>.json` .

3	4	1
CLEARED	HELD_FOR_REVIEW	REJECTED

LIVE WALKTHROUGH

Demo Flow

- `pip install -r requirements.txt` inside a Python 3.12 venv.
- `python orchestrator.py` — process all 8 sample transactions; show the printed summary (3 CLEARED, 4 HELD_FOR_REVIEW, 1 REJECTED).
- `python -m http.server 8000` from repo root, open `http://localhost:8000/frontend/` — live dashboard reading `shared/results/`.
- `python -m pytest --cov=pipeline --cov=mcp --cov-report=term-missing` — 69 passed, 99% coverage.
- Open the repo in an MCP-aware client (`.mcp.json` auto-starts `context7` and `pipeline-status`) and call `get_transaction_status` , `list_pipeline_results` , read `pipeline://summary` live.
- (Optional) Trigger the coverage-gate hook via `git push` and show it pass at 99% coverage.

TRUST & SAFETY

Security & Compliance by Design

- `Decimal`-only money path — no `float` anywhere on amount comparisons or scoring math.
- Offline, deterministic currency and watchlist checks — no runtime network dependency for compliance-critical logic.
- PII discipline enforced at multiple layers:
  - `pipeline/common.py`'s `append_audit_log` only accepts `(stage, transaction_id, outcome)` by construction.
  - `mcp/server.py` strips `source_account` / `destination_account` / `description` from every tool response.
  - `frontend/app.js` never renders those fields either.

RETROSPECTIVE

Lessons Learned

- A strict file-based, envelope-driven inter-stage protocol (vs. direct function calls) made per-record failure isolation and independent testing of each stage straightforward, at the cost of more orchestration bookkeeping in `orchestrator.py`.
- Writing the domain rules (Decimal-only, offline ISO 4217, PII masking) into `agents.md` / `specification.md` up front meant the generated pipeline code needed very little rework during test-writing — most edge cases (e.g. `TXN003`'s watchlist hold overriding its low fraud score) were already anticipated by the spec.
- Keeping the front-end and MCP server strictly read-only consumers of `shared/results/` simplified reasoning about data flow and kept both surfaces trivially safe to demo.

THANK YOU

Q&A / Links

- `README.md` — architecture, stage responsibilities, tech stack.
- `HOWTORUN.md` — exact commands for setup, pipeline run, dashboard, tests, MCP.
- `specification.md` / `agents.md` — full technical spec and domain rules.
- `research-notes.md` — context7 queries made during Task 2/4.

Python 3.12 **FastMCP** **pytest · 99% coverage** **context7**